

Apresentação em 01/10

Finanças Pessoais · <https://dsai-ap1-financas.vercel.app> · repositório github.com/polysamo/dsai-ap1-financas · sem slides: o repositório e a aplicação bastam

1. A URL ao vivo: o fluxo principal (cerca de 3 min)

- Abrir a URL e fechar o tour de boas-vindas (ou usar "Pular").
- Configurações → **Carregar dados de exemplo**: contas, quatro meses de histórico, orçamento, metas e recorrências aparecem de uma vez.
- **Dashboard**: receitas, despesas, resultado e saldo do mês; despesas por categoria; projeção dos próximos 6 meses.
- **Transações**: registrar uma despesa nova e ver saldo e totais mudarem. **Orçamento**: a barra da categoria muda de estado.
- **Cartões**: fatura, vencimento e uma compra parcelada. **Importar CSV**: prévia, linha com erro, duplicata e "Desfazer importação".
- **Relatórios** → Exportar CSV. Por último, ligar "Ocultar valores".

2. A spec em um minuto (1 min)

O que a aplicação faz

- App de finanças pessoais só no navegador: React, TypeScript, dados no `localStorage`, sem backend nem login.
- Núcleo (7 specs): visão geral, contas, categorias e transações, orçamento, metas, importação de CSV, dashboard e projeção.
- Extensões, cada uma com a própria spec: cartões e faturas, relatórios, calendário, dívidas, investimentos, tags e regras, configurações, ajuda e tour, transferências, orçamento anual, assinaturas, patrimônio, OFX, independência financeira, divisão de despesas, alertas, conciliação e fluxo de caixa.
- Regras fixas em todas: valores em centavos inteiros, datas AAAA-MM-DD, saldo de conta sempre calculado, projeção pela média dos 3 últimos meses completos mais recorrências.

O que ficou de fora

- Backend, sincronização entre dispositivos, login e criptografia dos dados.
- Várias moedas, Open Finance e integração bancária.
- Juros rotativos de cartão, PDF por biblioteca (usa a impressão do navegador) e XLSX.
- Subcategorias, edição em lote, busca global e PWA offline.
- Amortização extra que recalcula a tabela de dívidas (hoje ela abate só o saldo devedor).

3. Três prompts (cerca de 3 min)

O que melhor funcionou: "uma spec por parte, antes de qualquer código"

- O prompt pedia, para cada parte, "O quê e por quê", 8 a 12 critérios de aceitação verificáveis e "Fora do escopo", sem código.
- Por quê: cada critério virou um teste, quase um para um. Decisões como centavos inteiros e saldo calculado ficaram escritas uma vez e valeram em todo o código, sem retrabalho.

O que pior funcionou: "chegar a 100.000 linhas em 40 minutos"

- A meta de volume brigava com a regra do mesmo pedido: nada de enchimento, só código defensável. As duas não cabem no mesmo prazo.
- Resultado real: **23.722 linhas** (17.117 de aplicação e 6.605 de testes), não 100 mil. Até 16 agentes em paralelo geraram conflitos de merge, testes que estouravam o tempo na máquina carregada e criações de worktree que falharam.
- Lição: o número foi reportado como está. Medir o projeto por linhas incentiva volume, não valor.

O que mudou o projeto: "a projeção usa a média dos 3 meses mais as recorrências"

- A spec original deixava recorrências fora do escopo, e o prompt contrariava a spec escrita na própria sessão.
- Em vez de editar a spec antiga, foi criada a dashboard-e-projecao-v2, commitada antes do código, dizendo qual spec ela substitui. Depois vieram o painel de recorrências e a regra de "categorias cobertas", para não contar o mesmo valor duas vezes.
- Mais adiante, "não use mais Tailwind" levou à migração de todos os componentes para CSS puro, feita por cinco agentes em paralelo, e ao redesign da interface.

4. Os números (1 min)

Medida	Valor	Observação
Linhas de código (cloc)	23.722	17.117 de aplicação e 6.605 de testes; sem Markdown, JSON e dados. Meta de 100.000 não atingida.
Specs	26	Arquivos em SPEC/, incluindo a v2 do dashboard, que substitui a v1.
Prompts	11	Mensagens do usuário na sessão principal, contadas no registro da sessão.
Sessões	1 + 27	Uma sessão principal e 27 worktrees de subagentes (partes, migração de CSS e redesign).

Horas	cerca de 1h30	Das 16h14 às 17:43 de 01/10, com até 16 agentes ao mesmo tempo.
Commits	81	Trailers Agent: e Spec: em cada commit de parte.
Testes	516	Vitest e Testing Library, cobrindo cada critério de aceitação.

Números medidos às 17:43 de 01/10/2026, a partir do repositório e do registro da sessão. Os valores do README são os mesmos.